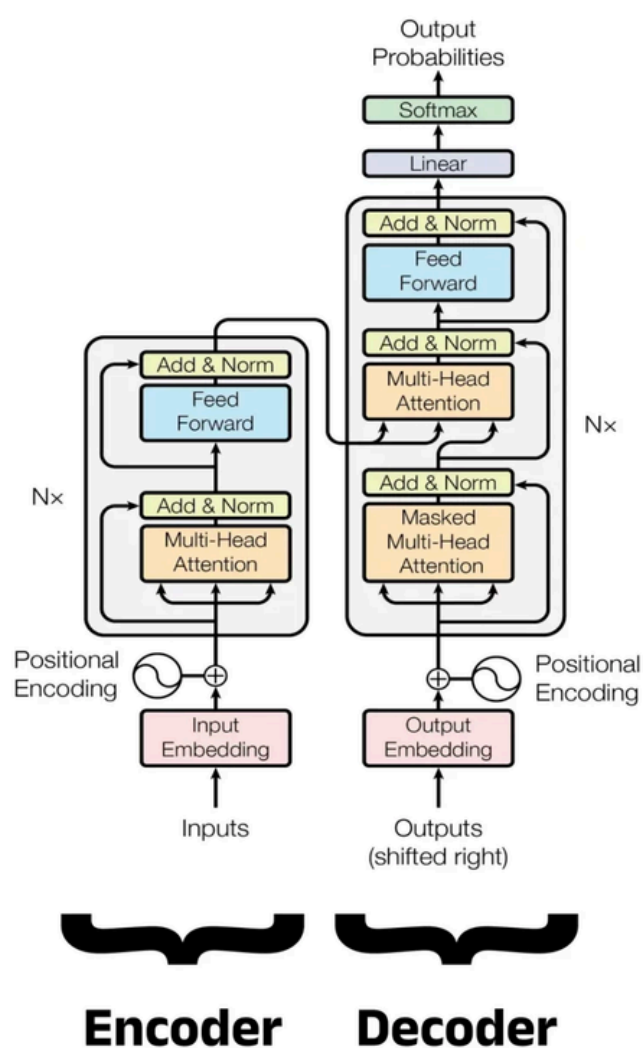
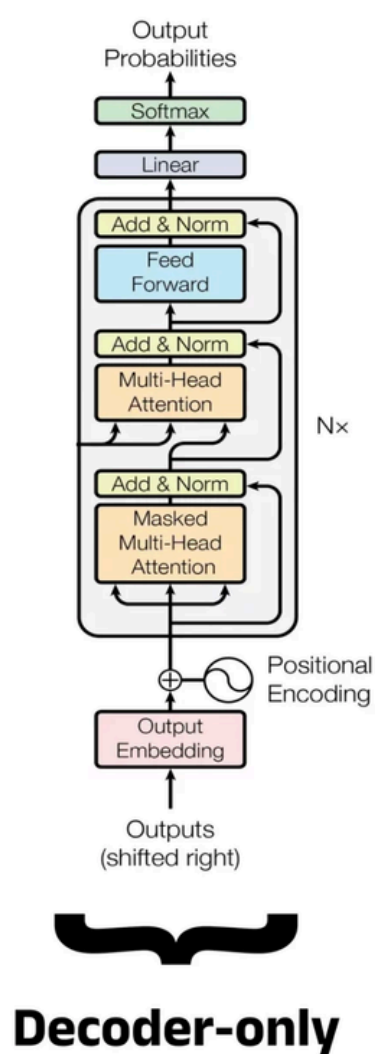


How Transformers Power LLMs

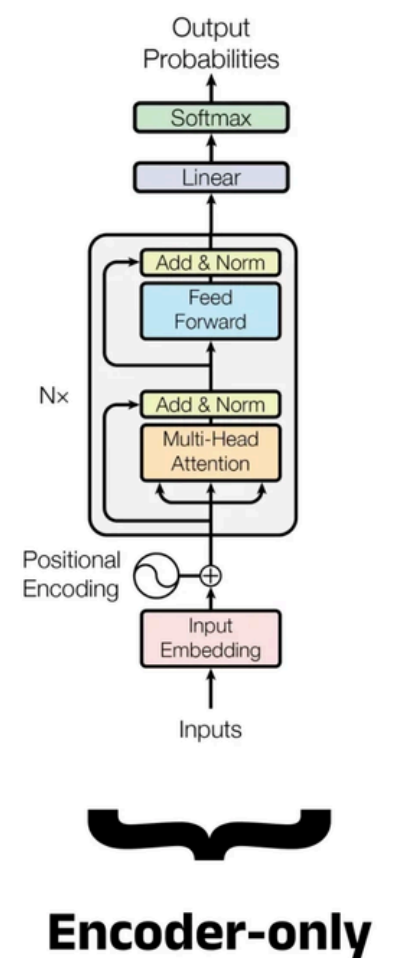
Transformer



GPT*



BERT*

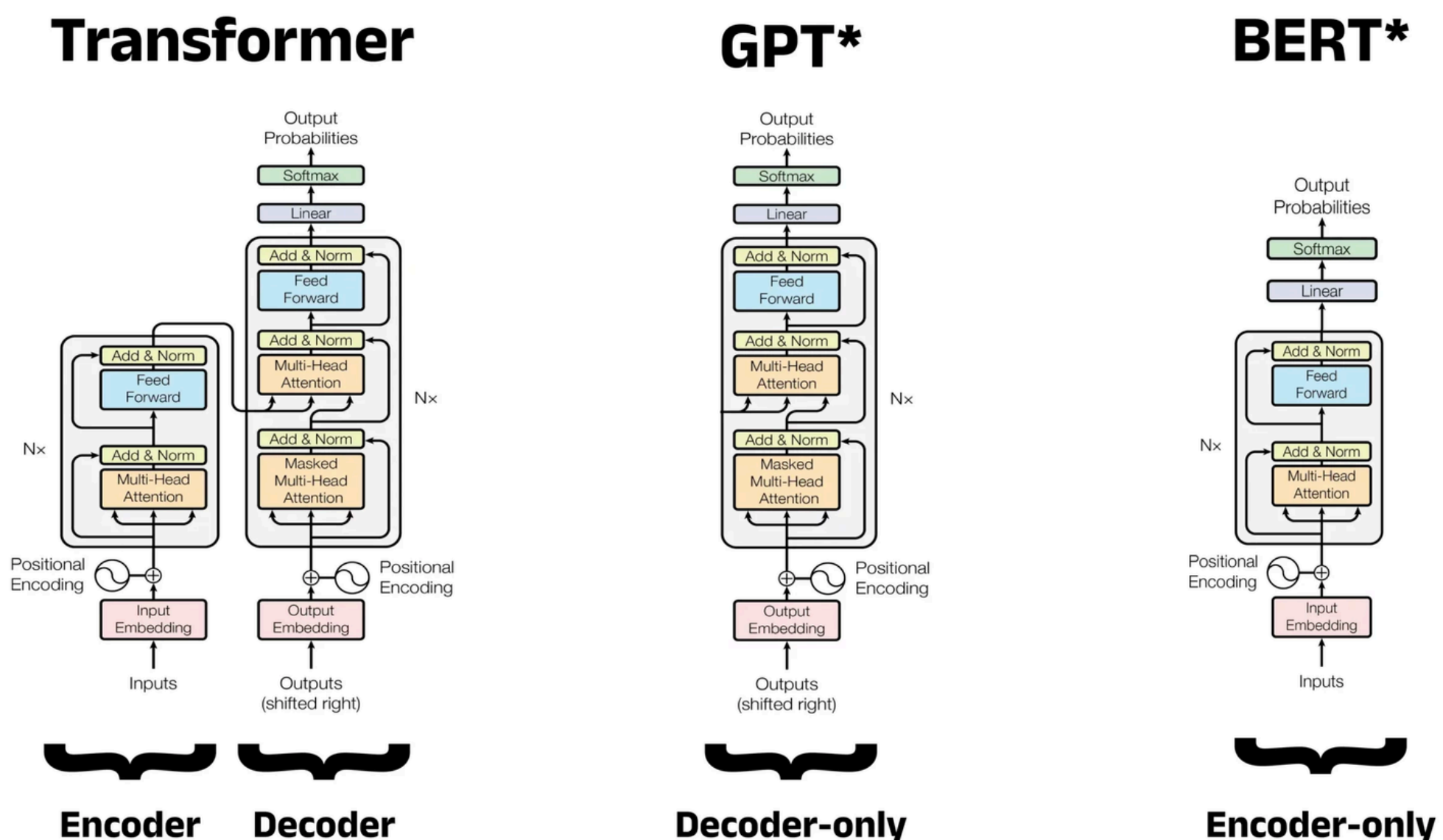


*Illustrative example, exact model architecture may vary slightly

How transformers power models like GPT, Claude, and Gemini

Modern AI systems use transformer architectures for their ability to handle large-scale language processing tasks. These models require large text datasets for training because they need to learn language patterns through specific modifications which meet their training needs.

The GPT models (GPT-4, **GPT-5**) use decoder-only Transformers i.e, a stack of decoder layers with masked self-attention. Claude (Anthropic) and **Gemini** (Google) also use similar transformer stacks, which they modify through their custom transformations. Google's Gemma models use the transformer design from the "Attention Is All You Need" paper to create text through a process which generates one token at a time.



*Illustrative example, exact model architecture may vary slightly

Part 1: How Text Becomes Machine-Readable

The first step toward transformer operation requires text conversion into numerical form for transformer processing. The process begins with tokenization and embeddings which require conversion of words into distinct tokens followed by conversion of those tokens into vector representation. The system needs positional encodings because they help the model understand how words are arranged in a sentence. In this section we break down each step.

Step 1: Tokenization: Converting Text into Tokens

At its core, an LLM cannot directly ingest raw text characters. Neural networks operate on numbers, not text. The process of tokenization enables the conversion of a complete text string into separate elements which receive individual numeric identifiers.

Why LLMs Cannot Understand Raw Text

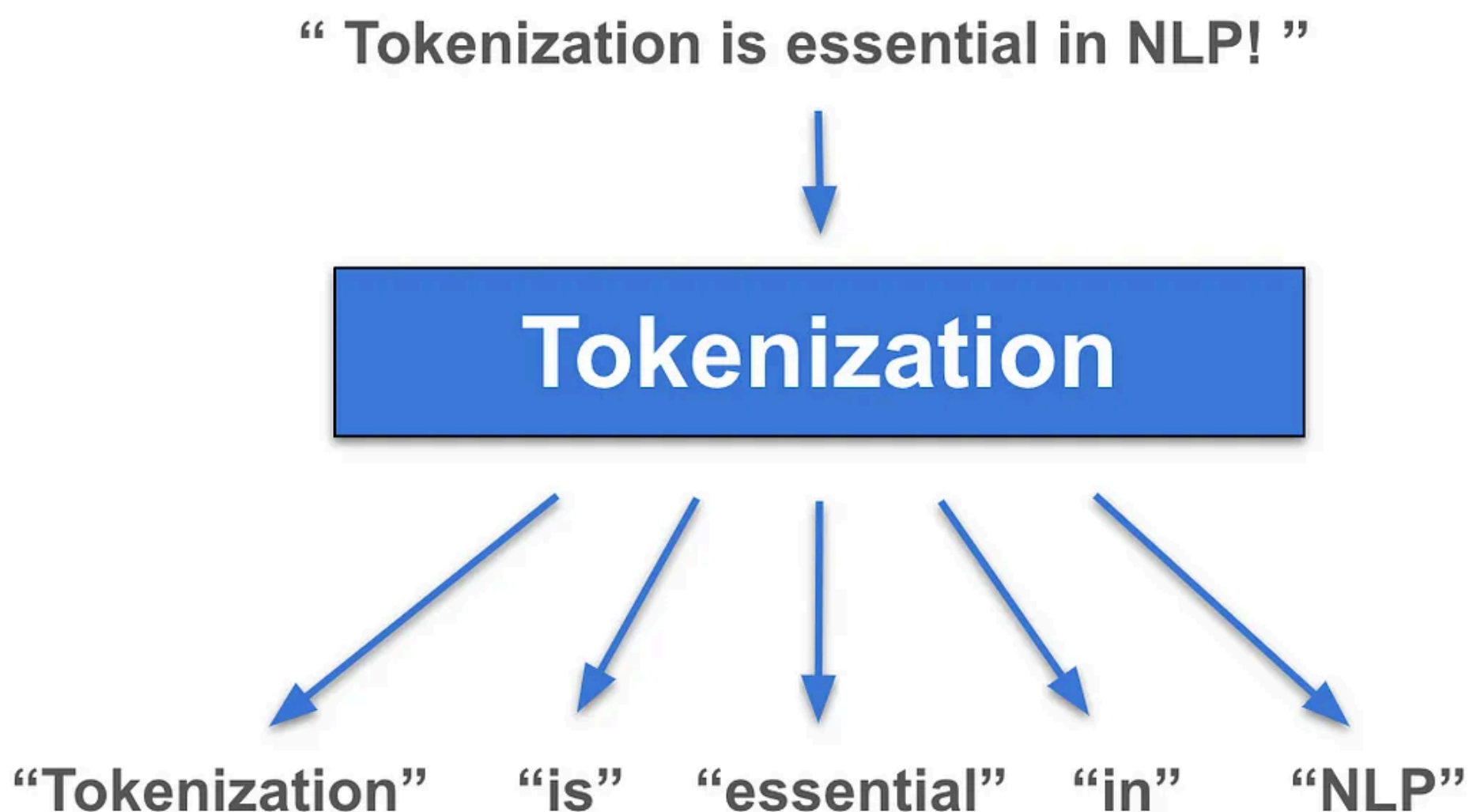
The model requires numeric input because raw text exists as a character string. We can't create a word-to-index mapping system because language contains infinite possible forms through its various tenses and plural forms and through the introduction of new vocabulary. The complete text of raw materials does not contain the necessary numerical framework that neural networks need for their mathematical computations. For example, the sentence: Transformers changed natural language processing

This must first be converted into a sequence of tokens before the model can process it.

How Tokenization Works

Tokenization segments text into smaller sections which correspond to linguistic components. The tokens can represent three different elements which include: words and subwords and characters and punctuation.

For example:



The model uses a unique numerical Id to represent each token which it needs for both training and inference purposes.

Types of Tokens Used in LLMs

Different tokenization strategies exist depending on the model architecture and vocabulary design. The methods include Byte-Pair Encoding (BPE), WordPiece, and Unigram. The methods maintain common words as single tokens while they divide uncommon words into essential components.

The word “transformers” remains whole while “unbelievability” breaks down into “un” “believ” “ability”. Subword tokenization enables models to process new or uncommon terms by using known word components. Tokenizers treat word pieces as basic units and special tokens (like “) and punctuation marks as distinct units.

Step 2: Token Embeddings: Turning Tokens into Vectors

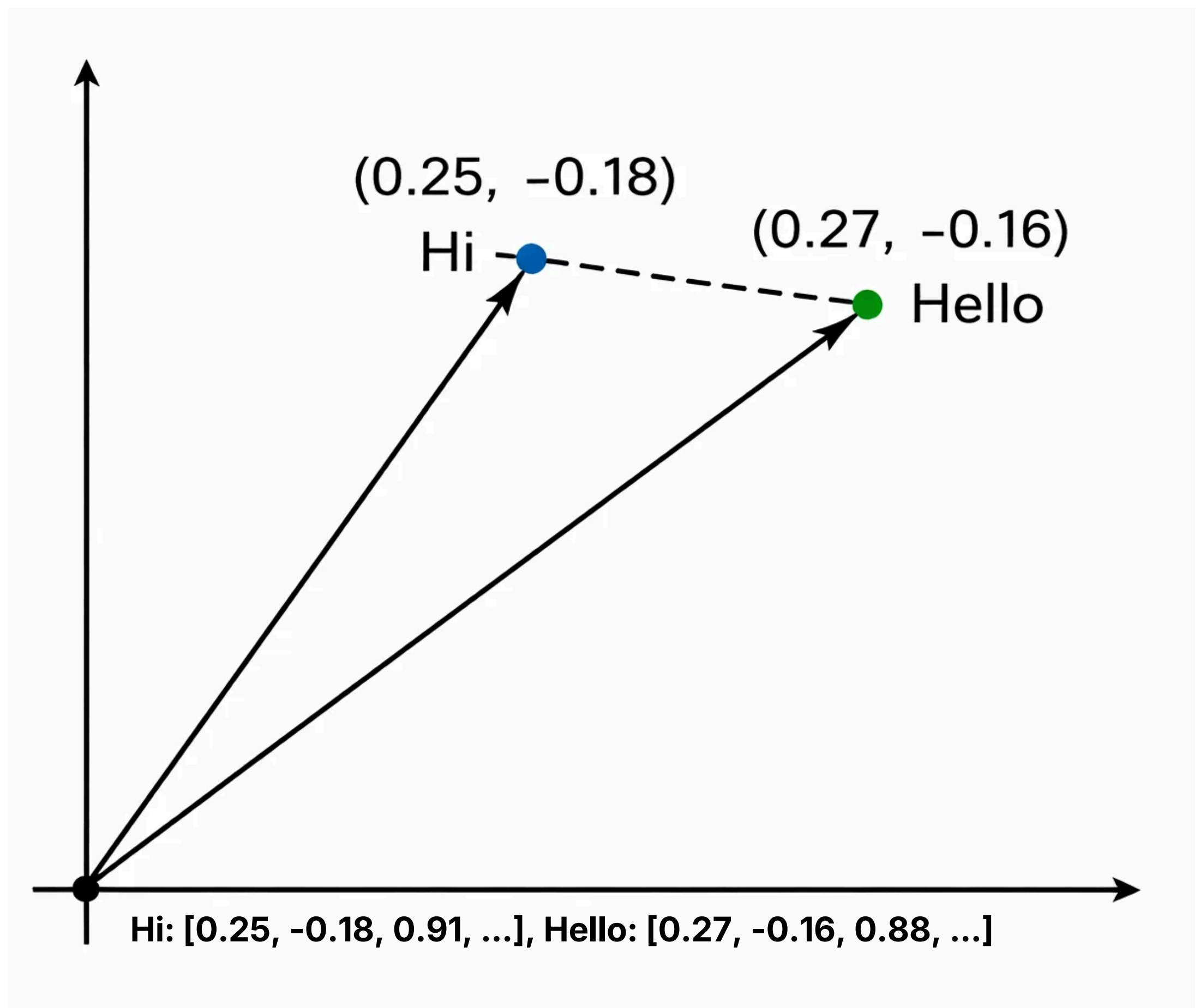
The model uses the acquired tokens to create an embedding vector for each token ID. The token embeddings represent word meaning through the use of dense numeric vectors.

An embedding is a numeric vector representation of a token. You can think of it as each token having coordinates in a high-dimensional space. The word “cat” will map to a vector that exists in 768 dimensions. The model acquires these embeddings through its training process. The tokens which have equivalent meanings produce vectors which show their relationship to one another. The words “Hello” and “Hi” have close embedding values but “Hello” and “Goodbye” show a large distance between their respective embeddings.

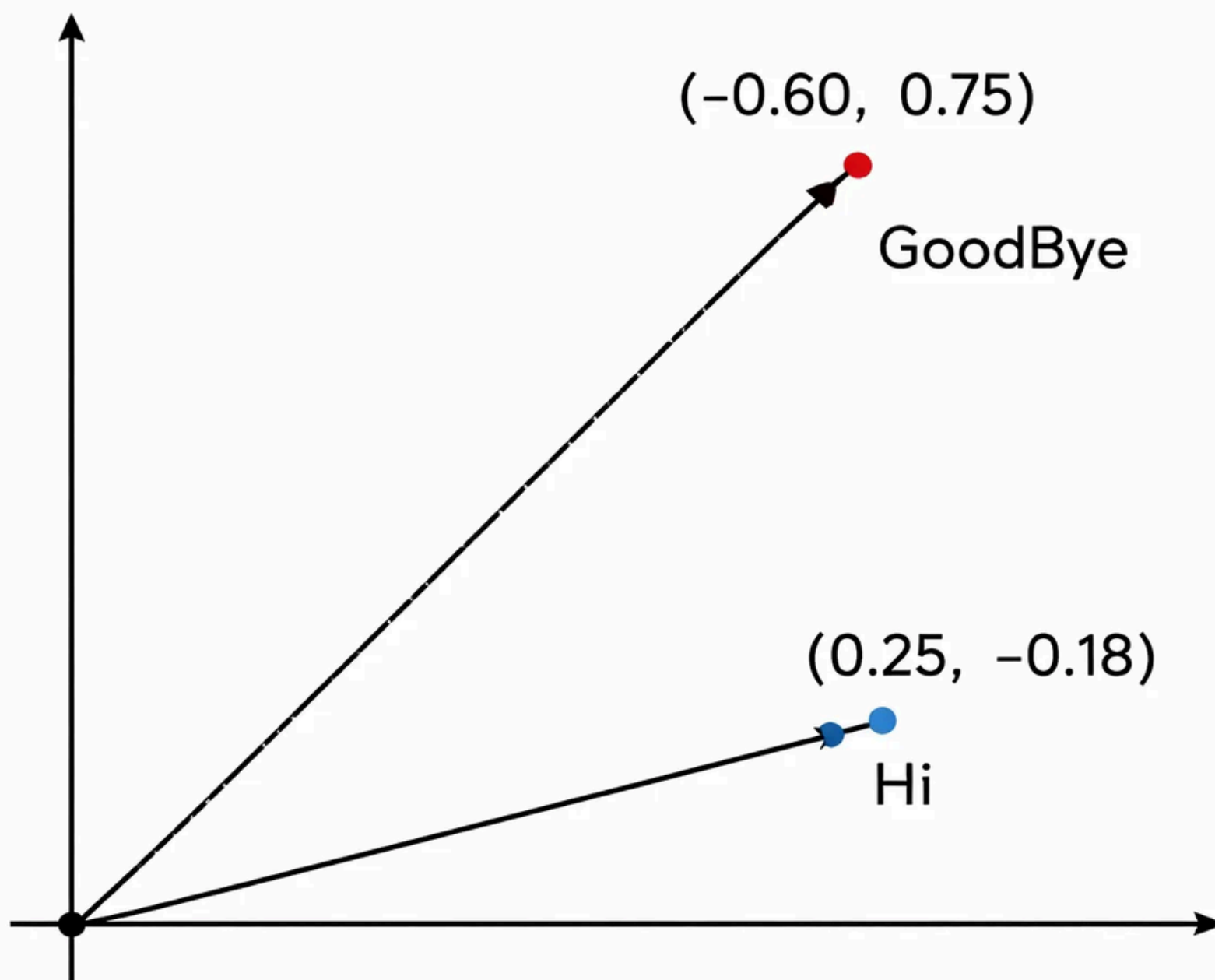
What is an Embedding?

The model uses the acquired tokens to create an embedding vector for each token ID. The token embeddings represent word meaning through the use of dense numeric vectors.

An embedding is a numeric vector representation of a token. You can think of it as each token having coordinates in a high-dimensional space. The word "cat" will map to a vector that exists in 768 dimensions. The model acquires these embeddings through its training process. The tokens which have equivalent meanings produce vectors which show their relationship to one another. The words "Hello" and "Hi" have close embedding values but "Hello" and "Goodbye" show a large distance between their respective embeddings.



Like here we can see that the embeddings of Hi and Hello are quite similar. And the embeddings of Hi, and GoodBye are quite distant to each other.



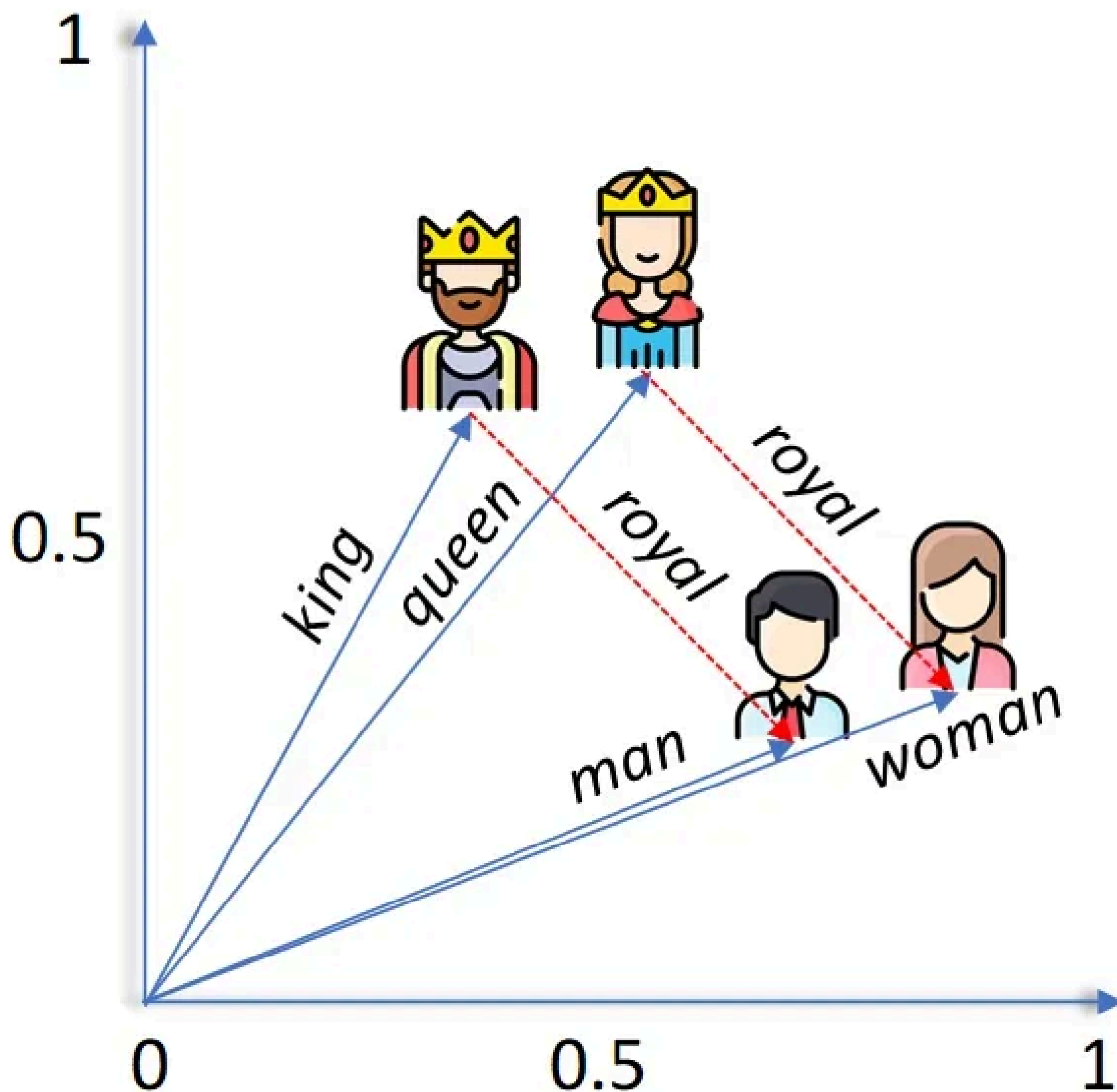
Hi: [0.25, -0.18, 0.91, ...], GoodBye: [-0.60, 0.75, -0.20, -0.55]

Semantic Meaning in Vector Space

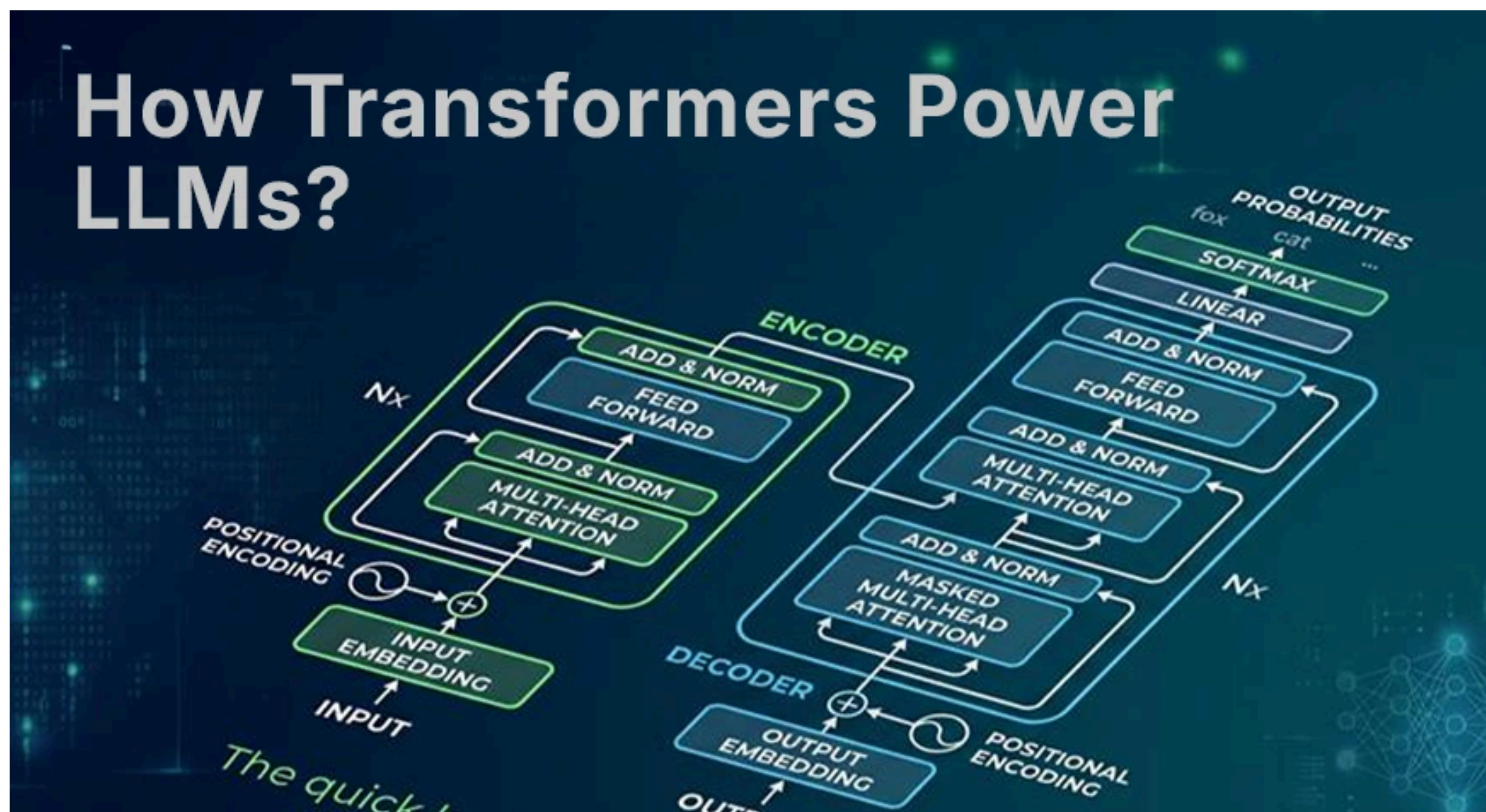
Embeddings capture meaning which enables us to assess relationships through vector similarity measurements. The vectors for "cat" and "dog" show closer proximity than those for "cat" and "table" because their semantic relationship is stronger. The model discovers word similarity through the initial stage of its processing. A token's embedding begins as a basic meaning which lacks context because it only shows the specific word meaning. The system first learns basic word meanings through its attention system which brings in context later on.

For example:

- The words king and queen show a pattern of appearing in close proximity.
- The two fruits apple and banana show a tendency to group together.
- The words car and vehicle demonstrate comparable spatial distributions in the environment.
- The spatial structure of the system enables training models to develop understanding of word connections



For more information, visit this article



Advanced

LLMs

Transformers

How Transformers Power LLMs: Step-by-Step Guide

A complete, step-by-step guide explaining how Transformer architecture powers modern LLMs like GPT and Gemini.